

Q1

```
C:\Users\hanpe>curl -X "POST" https://localhost:7289/api/J1/Delievddroid -H "Content-type:application/x-www-form-urlencoded" -d "Collisions=2&Deliveries=5"
730
C:\Users\hanpe>curl -X "POST" https://localhost:7289/api/J1/Delievddroid -H "Content-type:application/x-www-form-urlencoded" -d "Collisions=10&Deliveries=0"
-100
C:\Users\hanpe>curl -X "POST" https://localhost:7289/api/J1/Delievddroid -H "Content-type:application/x-www-form-urlencoded" -d "Collisions=3&Deliveries=2"
70
C:\Users\hanpe>
```

```
/// The score is computed using the following rules:
/// - Gain 50 points for each package delivered.
/// - Lose 10 points for each collision with an obstacle.
/// - Earn a bonus of 500 points if the number of packages delivered exceeds the number of collisions.
/// </summary>
/// <param name="Collisions">The number of collisions with obstacles.</param>
/// <param name="Deliveries">The number of packages successfully delivered.</param>
/// <returns>
/// An integer representing the final score after all calculations, including the bonus if applicable.
/// </returns>
/// <example>
/// POST: /api/J1/Delievddroid
/// Header: Content-Type: application/x-www-form-urlencoded
/// POST DATA: Collisions=2&Deliveries=5
/// -> 730
/// </example>
[HttpPost(template:"Delievddroid")]
[Consumes("application/x-www-form-urlencoded")]
0 references
public int Delievddroid([FromForm]int Collisions, [FromForm]int Deliveries)
{
    int del = 50 * Deliveries;
    int col = -10 * Collisions;
    int bonus;
    if (Deliveries > Collisions)
    {
        bonus = 500;
    }
    else
    {
        bonus = 0;
    }
    int final = del + col + bonus;
    return final;
}
```

Q2

<https://cemc.uwaterloo.ca/sites/default/files/documents/2022/2022CCCJrProblemSet.htm>

!

A regular box of cupcakes holds 8 cupcakes, while a small box holds 3 cupcakes. There are 28 students in a class and a total of at least 28 cupcakes. Your job is to determine how many cupcakes will be left over if each student gets one cupcake.

```
C:\Users\hanpe>curl -X "POST" https://localhost:7289/api/q2/ccleft -H "Content-type:application/x-www-form-urlencoded" -d "regularbox=3&smallbox=2"
There are 2 cupcakes left
C:\Users\hanpe>curl -X "POST" https://localhost:7289/api/q2/ccleft -H "Content-type:application/x-www-form-urlencoded" -d "regularbox=2&smallbox=2"
not enough
C:\Users\hanpe>curl -X "POST" https://localhost:7289/api/q2/ccleft -H "Content-type:application/x-www-form-urlencoded" -d "regularbox=-1&smallbox=0"
not valid number for boxes
C:\Users\hanpe>curl -X "POST" https://localhost:7289/api/q2/ccleft -H "Content-type:application/x-www-form-urlencoded" -d "regularbox=5&smallbox=0"
There are 12 cupcakes left
C:\Users\hanpe>
```

```

    /// </summary>
    /// <param name="regularbox">The number of regular boxes, each containing 8 cupcakes.</param>
    /// <param name="smallbox">The number of small boxes, each containing 3 cupcakes.</param>
    /// <returns>A message indicating either the number of leftover cupcakes or if there are not enough cupcakes.</returns>
    /// <example>
    /// POST: /api/q2/ccleft
    /// Header: Content-Type: application/x-www-form-urlencoded
    /// POST DATA: regularbox=3&smallbox=2
    /// -> "There are 2 cupcakes left"
    /// </example>
    [HttpPost(template: "ccleft")]
    [Consumes("application/x-www-form-urlencoded")]
    0 references
    public string ccleft([FromForm] int regularbox, [FromForm] int smallbox)
    {
        if (regularbox >= 0 && smallbox >= 0)
        {
            int regcake = 8 * regularbox;
            int smacake = 3 * smallbox;
            int total = regcake + smacake;
            int left = total - 28;
            if (left < 0)
            {
                return "not enough";
            }
            else
            {
                return "There are " + left + " cupcakes left";
            }
        }
        else
        {
            return "not valid number for boxes";
        }
    }
}

```

Q3

```

C:\Users\hanpe>curl https://localhost:7289/api/J2/ChiliPeppers?Ingredients=Poblano,Cayenne,Thai,Poblano
118000
C:\Users\hanpe>curl https://localhost:7289/api/J2/ChiliPeppers?Ingredients=Habanero,Habanero,Habanero,Habanero,Habanero
625000
C:\Users\hanpe>curl https://localhost:7289/api/J2/ChiliPeppers?Ingredients=Poblano,Mirasol,Serrano,Cayenne,Thai,Habanero
,Serrano
278500
C:\Users\hanpe>|

```

```

public int ChiliPeppers(string Ingredients)
{
    string[] subs = Ingredients.Split(",");
    int Poblano = 0;
    int Mirasol = 0;
    int Serrano = 0;
    int Cayenne = 0;
    int Thai = 0;
    int Habanero = 0;
    foreach (string s in subs)
    {
        if (s == "Poblano")
        {
            Poblano++;
        } else if (s == "Mirasol")
        {
            Mirasol++;
        } else if (s == "Serrano")
        {
            Serrano++;
        } else if (s == "Cayenne")
        {
            Cayenne++;
        } else if (s == "Thai")
        {
            Thai++;
        } else if (s == "Habanero")
        {
            Habanero++;
        }
    }
    int total = Poblano * 1500 + Mirasol * 6000 + Serrano * 15500 + Cayenne * 40000 + Thai * 75000 + Habanero * 125000;
    return total;
}

```

Q4

<https://cenc.uwaterloo.ca/sites/default/files/documents/2015/2015CCCJrProblems.html>

We often include emoticons in our text messages to indicate how we are feeling. The three consecutive characters `:-)` indicate a happy face and the three consecutive characters `:-(` indicate a sad face.

Write a program to determine the overall mood of a message.

The output is determined by the following rules:

If the input line does not contain any happy or sad emoticons, output `=> none`.

Otherwise, if the input line contains an equal number of happy and sad emoticons, output `=> unsure`.

Otherwise, if the input line contains more happy than sad emoticons, output `=> happy`.

Otherwise, if the input line contains more sad than happy emoticons, output `=> sad`.

```
C:\Users\hanpe>curl https://localhost:7289/api/q4/mood?input=How%20are%20you%20%3A-%29%20doing%20%3A-%28%20today%20%3A-%29%3F
happy
C:\Users\hanpe>curl https://localhost:7289/api/q4/mood?input=%3A%29
none
C:\Users\hanpe>curl https://localhost:7289/api/q4/mood?input=This%3A-%28is%20str%3A-%28%3A-%28ange%20te%3A-%29xt.
sad
C:\Users\hanpe>curl https://localhost:7289/api/q4/mood?input=choose%20%3A-%29%20or%20%3A-%28
unsure
C:\Users\hanpe>
```

```
/// "none" if there are no emoticons,
/// "unsure" if there are equal numbers of happy and sad emoticons,
/// "happy" if there are more happy emoticons,
/// or "sad" if there are more sad emoticons.
/// </returns>
/// <example>
/// GET: api/q4/mood?input=How are you :-) doing :-( today :-)?
/// -> "happy"
/// GET: api/q4/mood?input=:)
/// -> "none"
/// GET: api/q4/mood?input=This:-(is str:-(:(ange te:-(xt.
/// -> "sad"
/// GET: api/q4/mood?input=choose :-) or :-(
/// -> "unsure"
/// </example>
[HttpGet(template: "mood")]
0 references
public string mood(string input)
{
    string emo1 = ":-)";
    string emo2 = ":-(";

    int happy = input.Split(emo1).Length - 1;
    int sad = input.Split(emo2).Length - 1;

    if (happy == 0 && sad == 0)
    {
        return "none";
    } else if (happy == sad)
    {
        return "unsure";
    } else if (happy > sad)
    {
        return "happy";
    } else return "sad";
}
}
```

Q5

<https://cemc.uwaterloo.ca/sites/default/files/documents/2024/2024CCCJrProblems.html>

After completing a competition, you are struck with curiosity. How many participants were awarded bronze level?

Gold level is awarded to all participants who achieve the highest score. Silver level is awarded to all participants who achieve the second highest score. Bronze level is awarded to all participants who achieve the third highest score.

Given a list of all the scores, your job is to determine the score required for bronze level and how many participants achieved this score.

```
C:\Users\hanpe>curl https://localhost:7289/api/J3/bronze?scores=70,62,58,73
The score required for bronze level is 62 and 1 participant achieved this score.
C:\Users\hanpe>curl https://localhost:7289/api/J3/bronze?scores=75,70,60,70,70,60,75,70
The score required for bronze level is 60 and 2 participant achieved this score.
C:\Users\hanpe>curl https://localhost:7289/api/J3/bronze?scores=75,70
Not enough unique scores to determine the bronze level.
C:\Users\hanpe>
```

```
/// -> "The score required for bronze level is 60 and 2 participant achieved this score."
/// </example>
[HttpGet(template: "bronze")]
0 references
public string bronze(string scores)
{
    string[] scoreStrings = scores.Split(',');
    int[] scoreArray = new int[scoreStrings.Length];
    int scoreIndex = 0;

    foreach (string scoreString in scoreStrings)
    {
        int score;
        if (int.TryParse(scoreString, out score))
        {
            scoreArray[scoreIndex] = score;
            scoreIndex++;
        }
    }

    int[] validScores = new int[scoreIndex];
    for (int i = 0; i < scoreIndex; i++)
    {
        validScores[i] = scoreArray[i];
    }

    int[] uniqueScores = validScores.Distinct().OrderByDescending(s => s).ToArray();
    if (uniqueScores.Length < 3)
    {
        return "Not enough unique scores to determine the bronze level.";
    }

    int bronzeScore = uniqueScores[2];
    int participantsWithBronze = validScores.Count(s => s == bronzeScore);

    return "The score required for bronze level is " + bronzeScore + " and " + participantsWithBronze + " participant achieved this score.";
}
```